树形动态规划（Tree DP）

一、基本概念与特点

1.1 什么是树形DP

树形动态规划是在**树结构**上进行的动态规划，通过递归遍历树的节点，**自底向上**或**自顶向下**地计算状态，解决与树结构相关的优化问题。

1.2 核心特点

1. **树结构**：问题模型是树或森林
2. **递归性质**：子树具有相似结构，可递归求解
3. **无后效性**：父节点状态依赖于子节点，子节点不影响祖先
4. **方向性**：通常以某节点为根的子树作为状态空间

1.3 基本思想

分治思想在树上的应用：将整棵树的问题分解为各子树的问题，合并得到原问题的解。

二、树形DP基本操作

- 把树转为有根树(如果是几个互不连通的树，就加一个虚拟根，它连接所有孤立的树)
- 在树上做DFS，递归到最底层的叶子结点，再一层层返回信息更新至根结点。
- 树上DP所操作的就是这一层层返回的信息。

2.1 状态设计

常用状态定义形式：

- `dp[u]`：以u为根的子树的信息
- `dp[u][j]`：以u为根的子树，容量/数量限制为j时的最优解
- `dp[u][0/1]`：u节点选/不选时的最优解

2.2 转移方程

基于树的递归结构：

```
dp[u] = combine(dp[v1], dp[v2], ..., dp[vk])  // v是u的子节点
```

2.3 例题 二叉苹果树 洛谷P2015

题目描述

有一棵苹果树，如果树枝有分叉，一定是分二叉（就是说没有只有一个儿子的结点）

这棵树共有 N 个结点（叶子点或者树枝分叉点），编号为 $1 \sim N$，树根编号一定是 1。

我们用一根树枝两端连接的结点的编号来描述一根树枝的位置。下面是一颗有 4 个树枝的树：

```
2   5
 \ /
  3   4
   \ /
    1
```

现在这颗树枝条太多了，需要剪枝。但是一些树枝上长有苹果。

给定需要保留的树枝数量，求出最多能留住多少苹果。

输入格式

第一行 2 个整数 N 和 Q，分别表示表示树的结点数，和要保留的树枝数量。

接下来 $N-1$ 行，每行 3 个整数，描述一根树枝的信息：前 2 个数是它连接的结点的编号，第 3 个数是这根树枝上苹果的数量。

输出格式

一个数，最多能留住的苹果的数量。

输入输出样例 #1

输入 #1

```
5 2
1 3 1
1 4 10
2 3 20
3 5 20
```

输出 #1

```
21
```

说明/提示

$1 \leqslant Q < N \leqslant 100$，每根树枝上的苹果 $\leqslant 3 \times 10^4$。

2.3.1 DP状态和DP状态转移方程

定义状态dp[u][j]，表示以结点u为根的子树上留j条边时的最多苹果数量。dp[1][q]就是答案。

v是u的一个子结点。dp[u][j]的计算:

- dp[v][k]:在v上留k条边;
- dp[u][j-k-1]:除了v上的k条边，以及边[u,v]，以u为根的这棵树上还有j-k-1个边，它们在u的其他子结点上。

2.3.2 参考代码

```
//洛谷P2015的代码（邻接表存树）
#include<bits/stdc++.h>
using namespace std;
const int N = 200;
struct node{
int v, w;                    //v是子结点，w是边[u,v]的值
node(int v = 0, int w = 0) : v(v), w(w) {}
};
vector<node> edge[N];
int dp[N][N], sum[N];  //sum[i]记录以点i为根的子树的总边数
int n, q;
void dfs(int u, int father){
        for(int i = 0; i < edge[u].size(); i++) {  //用i遍历u的所有子结点
        int v = edge[u][i].v, w =  edge[u][i].w;
        if(v == father) continue;              //不回头搜父亲，避免循环
        dfs(v, u);                             //递归到最深的叶子结点，然后返回
        sum[u] += sum[v] + 1;                  //子树上的总边数
      //for(int j=sum[u];j>=0;j--)
      //    for(int k=0;k<=j-1;k++)            //两个for优化为下面的代码。不优化也行
        for(int j = min(q, sum[u]); j >= 0; j--)
            for(int k = 0; k <= min(sum[v], j - 1); k++)
                dp[u][j] = max(dp[u][j], dp[u][j-k-1] + dp[v][k] + w);
    }
}
int main(){
    scanf("%d%d", &n, &q);            //n个点，留q条树枝
for(int i = 1; i < n; i++){
        int u, v, w;  scanf("%d%d%d", &u, &v, &w);
        edge[u].push_back(node(v,w)); //把边[u,v]存到u的邻接表中
        edge[v].push_back(node(u,w)); //无向边
    }
    dfs(1, 0);                        //从根结点开始做记忆化搜索
    printf("%d\n", dp[1][q]);
    return 0;
}
```

三、背包与树形DP

上面的题目“二叉苹果树”，可以建模为“分组背包”:

- (1)分组。根结点u的每个子树是一个分组。
- (2)背包的容量。把u为根的整棵树上的树枝数，看成背包容量。
- (3)物品。把每个树枝看成一个物品，体积为1，树枝上的苹果数量看成物品的价值。
- (4)背包目标。求能放到背包的物品的总价值最大，就是求留下树枝的苹果数最多。

树形背包问题的状态定义，一般用dp[u][j]表示以点u为根的子树中，选择j个点(或j个边)的最优情况。

树形DP是解决树结构问题的强大工具，掌握常见模型和技巧后，能够解决许多复杂的优化问题。通过大量练习，可以培养出快速识别问题类型并设计解决方案的能力。